

Advanced Git Concepts

This document explains two important advanced Git concepts used in professional development workflows:

1. Stashing changes (git stash)
2. Rebasing vs Merging

Each concept is explained with definition, purpose, and practical command based steps so the workflow can be directly practiced.

1. Stashing Changes (git stash)

Definition

Git stash temporarily saves uncommitted changes so that the working directory becomes clean. This allows you to switch branches or perform other operations without committing incomplete work.

Stashing is useful when you are working on a task but suddenly need to switch to another branch to fix a bug or make an urgent update.

When to Use git stash

- When work is incomplete but you must change branches
 - When you want to temporarily store modifications
 - When the working directory must be cleaned quickly
-

Basic Stash Workflow

Step 1: Check current changes

```
git status
```

This shows modified or untracked files.

Step 2: Save changes using stash

```
git stash
```

Git stores the changes and restores the working directory to the last committed state.

Step 3: Verify working directory

```
git status
```

The repository should now appear clean.

Step 4: View stored stashes

```
git stash list
```

Example output:

```
stash@{0}: WIP on main  
stash@{1}: WIP on feature
```

Restoring Stashed Changes

Apply the most recent stash

```
git stash apply
```

This restores the changes but keeps the stash stored.

Apply and remove stash

```
git stash pop
```

This restores the changes and removes the stash entry.

Apply a specific stash

```
git stash apply stash@{0}
```

Deleting Stashes

Remove a specific stash

```
git stash drop stash@{0}
```

Clear all stashes

```
git stash clear
```

Practical Example

```
git status
git stash

git checkout bug-fix-branch

# perform bug fix work

git checkout main

git stash pop
```

The original work is restored after finishing urgent work.

2. Rebasing vs Merging

Both rebasing and merging integrate changes from one branch into another branch.

However, the way history is handled is different.

Merging

Definition

Merging combines changes from one branch into another by creating a merge commit. The full commit history of both branches is preserved.

Merge Workflow

Step 1: Switch to main branch

```
git checkout main
```

Step 2: Merge feature branch

```
git merge feature-branch
```

Git combines the changes and creates a merge commit.

Resulting History

The history remains non linear with a merge commit joining both branches.

Rebasing

Definition

Rebasing moves the commits of one branch on top of another branch. This creates a cleaner, linear project history.

Rebase Workflow

Step 1: Switch to feature branch

```
git checkout feature-branch
```

Step 2: Rebase onto main branch

```
git rebase main
```

Git replays the feature branch commits on top of the main branch.

Resulting History

History becomes linear without merge commits.

Handling Rebase Conflicts

If conflicts occur during rebase:

Step 1: Resolve the conflict in files

Step 2: Continue rebase

```
git add .  
  
git rebase --continue
```

If necessary, cancel the rebase

```
git rebase --abort
```

Key Differences

Merging

- Keeps complete branch history
- Creates merge commits
- Easier for beginners

Rebasing

- Produces linear history
- Rewrites commit history
- Used for cleaner project logs

Professional Workflow Example

Feature development workflow

```
git checkout -b feature-login  
  
# make changes  
  
git add .  
  
git commit -m "Implemented login feature"  
  
# update branch with latest main  
  
git fetch origin  
  
git rebase origin/main  
  
# push branch  
  
git push origin feature-login
```

Summary

Stashing allows temporary storage of uncommitted work.



Merging combines branch histories while preserving commits.

Rebasing creates a clean and linear history by replaying commits.

Understanding these concepts helps maintain organized and professional Git workflows.

Prof.Pranjul Shukla

